

CH11 React 一复习要点

1 React 简介

1.1 React 的由来

[p.4]

- > 2011 年 Facebook 新闻动态页面复杂化，面临三大挑战 [p.4]:
 1. 代码复杂度急剧上升：大量 DOM 操作与业务逻辑混杂，难以维护
 2. 数据同步困难：多个组件共享状态时难以保持数据一致性
 3. 性能问题：频繁 DOM 更新导致页面卡顿
- > 工程师 **Jordan Walke** 创建 React，首次用于 Facebook 新闻流，**2013 年**开源 [p.4]

1.2 React 的核心特点

[p.5–6]

- > **React**：用于构建用户界面的 JavaScript 库 (Library) [p.5]
- > 设计理念 [p.5]:
 - **Learn Once, Write Anywhere**: Web、React Native（移动端）、Electron（桌面）
 - 组件化 (Component-based): UI 拆分为独立可复用组件
 - 状态驱动视图: UI 是状态的函数 $f(state) = UI$ ，状态变化自动更新视图
- > 核心定位：专注视图层，声明式描述 UI，可与任意后端技术栈配合 [p.5]

1.3 React 五大核心特性

[p.6]

特性	描述	解决的问题
声明式编程	描述 UI 状态，React 自动处理 DOM 更新	避免手动 DOM 操作，代码更易理解
组件化	UI 拆分为独立、可复用组件	提高复用性，便于团队协作
虚拟 DOM	高效 DOM 更新策略，减少直接操作	提升性能，避免频繁重排重绘
单向数据流	数据流向清晰可预测	避免数据混乱，便于调试追踪
JSX	JS 中编写类 HTML 结构	提升开发效率和代码可读性

1.4 库 vs 框架

[p.7–8]

- > React 作为库的特点 [p.7]: 只专注视图层、不强制项目结构、不提供路由/状态管理/HTTP 等完整方案
- > 灵活的技术选型 [p.7]: 路由 (React Router/Next.js)、状态管理 (Context/Redux/Zustand/Jotai)、HTTP (axios/fetch/SWR/React Query)
- > 无侵入性：可逐步引入现有项目，可与其他框架共存 [p.7]

维度	库 (Library) — React	框架 (Framework) — Angular
控制权	开发者调用库的 API	框架控制执行流程
约束程度	低约束，灵活使用	高约束，遵循约定（强制 TypeScript 等）
关注点	解决特定问题（视图层）	提供完整解决方案（路由 + 状态 + HTTP）
扩展性	易于替换和组合	难以替换核心模块

2 React 核心概念详解

2.1 JSX 语法

[p.10–14]

- > **JSX** (JavaScript XML) [p.11]: JavaScript 扩展语法，允许在 JS 中编写类 HTML 结构
- > 被 Babel 编译器转换为普通 JS 函数调用，不是浏览器原生支持 [p.11]
- > 编程范式 [p.11]: **命令式** (Imperative: ” 怎么做”) → **声明式** (Declarative: ” 做什么”)
- > **JSX 五项核心规则** [p.12]:
 1. 必须有**根元素包裹**（只能返回一个根元素）
 2. 标签必须闭合，自闭合标签用 `</>`
 3. 使用 `className` 替代 `class`（`class` 是 JS 关键字）
 4. 使用 `htmlFor` 替代 `for`（`for` 是 JS 关键字）
 5. `{ }` 大括号内写任意 JavaScript 表达式
- > JSX 表达式用法举例 [p.13]: 变量插值、条件渲染、列表渲染等
- > 编译过程 [p.14]: `JSX` → `Babel` → `React.createElement()` → 虚拟 DOM

2.2 组件化开发

[p.16–20]

- > 将复杂 UI 拆分为独立、可复用的组件 [p.16]
- > 五大优势 [p.16]: 高复用性、低耦合、易测试、可维护性、团队协作
- > **函数组件** [p.17]:
 - 必须以**大写字母**开头，使用 `PascalCase`（大驼峰命名）
 - 文件名与组件名一致
- > **Props** (Properties) [p.18–19]:
 - 组件间传递数据的方式，**从父组件流向子组件**（单向）
 - **只读性**: 子组件不能修改 props，保证数据单向性 [p.19]
 - 支持类型检查 (`PropTypes` / `TypeScript`) 和默认值设置 [p.19]
- > 子传父: 通过**回调函数**向父组件传递数据 [p.20]

2.3 状态管理—useState

[p.22–25]

- > 为什么需要 **State** [p.22]: 普通变量变化不会触发组件重新渲染，UI 无法自动更新
- > **useState Hook** [p.23]:
 - State 变化时 React **自动重新渲染**组件（响应式）
 - State 是组件**私有**的内部数据，外部无法直接访问
 - **不可直接修改** State，必须通过 `setState` 函数更新
 - 多个 State 更新会**批量处理** (Batching)，提高性能（异步更新） [p.23]
- > 语法: `const [state, setState] = useState(initialValue)` [p.23]
- > 使用范例和异步更新注意 [p.24–25]

2.4 副作用—useEffect

[p.27–31]

- > **useEffect** (副作用 Hook) [p.27]: 处理与组件渲染无关的操作
- > 常见副作用 [p.27]: 数据获取 (API 调用)、订阅事件 (WebSocket)、DOM 操作、定时器 (setTimeout/setInterval)、日志记录
- > 三种依赖数组模式 [p.28]:

依赖数组	执行时机	适用场景
[] 空数组	仅在组件挂载时执行一次	初始化操作、一次性数据获取
[dep] 有依赖	挂载时 + 每次 dep 变化	依赖特定数据的操作
不写 (无依赖数组)	每次组件渲染都执行	需同步更新的操作

- > **useState vs useEffect** 总结 [p.31]:
 - useState: 管理状态数据, 存储和更新组件内部数据
 - useEffect: 处理副作用, 管理副作用的执行时机

2.5 虚拟 DOM

[p.33–34]

- > 虚拟 DOM 是真实 DOM 的 **JavaScript 对象表示** (内存中的 DOM 树对象) [p.33]
- > 为什么需要 [p.33]: 直接操作真实 DOM 非常昂贵, 每次操作触发重排 (Reflow) 和重绘 (Repaint)
- > 四步工作原理 [p.33]:
 1. 首次渲染: JSX → 虚拟 DOM 树 → 真实 DOM
 2. 状态更新: 创建新的虚拟 DOM 树
 3. **Diff** 算法: 高效对比新旧虚拟 DOM 树差异
 4. 批量更新: 计算最小 DOM 更新操作, 批量应用到真实 DOM
- > **Diff** 算法三大策略 [p.34]:
 - 同层比较: 只比较同一层级节点, 不跨层
 - 类型不同则替换: 元素类型改变时直接重建子树
 - **key** 属性优化: 列表对比时用 key 识别节点身份

2.6 最佳实践

[p.35]

- > 单一职责原则: 每个组件只负责一个功能
- > 状态提升 (Lifting State Up): 共享状态提升到最近公共父组件
- > 代码组织规范: 组件文件、样式文件、测试文件分离
- > 命名规范: PascalCase 组件名, camelCase 方法名
- > 导入顺序: React → 第三方库 → 本地组件 → 样式

3 MPA 与 SPA 架构

[p.37–38]

- > **MPA** (Multi-Page Application) [p.37]:
 - 传统 Web 架构, 每个页面是独立 HTML 文件
 - 导航流程: 点击链接 → 浏览器请求 → 服务器返回新 HTML → 重新渲染整页
- > **SPA** (Single-Page Application) [p.37]:
 - 现代架构, 只有一个 HTML, JS 动态更新内容
 - 首次访问加载唯一 HTML → 后续导航通过 JS 处理 → 仅局部更新

> SPA 三大核心技术 [p.37]：前端路由 (React Router)、虚拟 DOM、状态管理 (Redux/Context)

维度	MPA	SPA
页面加载	每次导航重新加载整页	首次后仅局部更新
用户体验	有页面刷新感	流畅无刷新
首屏速度	较快（只加载当前页）	较慢（需加载完整应用）
SEO	天然友好	需 SSR/SSG 额外处理
开发复杂度	较低	较高（路由、状态管理等）
资源复用	低（重复加载公共资源）	高（一次加载多次复用）
适用场景	简单/内容型网站	复杂/交互型应用

4 本章小结

[p.40]

- > **React** 简介 [p.4–8]：Facebook 2013 开源，视图层库，声明式 + 组件化 + 状态驱动，库 vs 框架
- > **JSX** [p.10–14]：声明式 UI 描述语法，五项核心规则，编译为 `React.createElement`
- > **组件化** [p.16–20]：函数组件 (PascalCase)、Props 单向只读、回调传子 → 父
- > **useState** [p.22–25]：响应式状态管理，`setState` 异步批量更新
- > **useEffect** [p.27–31]：副作用处理，三种依赖数组模式
- > **虚拟 DOM** [p.33–34]：四步原理 + Diff 三策略（同层比较/类型替换/key 优化）
- > **MPA vs SPA** [p.37–38]：加载方式、SEO、首屏、适用场景对比

高频考点：(1) React 五大核心特性 [p.6]；(2) 库 vs 框架的核心区别（控制权、约束程度、关注点）[p.7]；(3) JSX 五项核心规则（根元素/闭合/className/htmlFor/大括号）[p.12]；(4) Props 只读性和单向数据流 [p.19]；(5) useState 三大特性（响应式/私有/setState 异步批量）[p.23]；(6) useEffect 三种依赖数组及执行时机 [p.28]；(7) 虚拟 DOM 四步原理 + Diff 三策略 [p.33-34]；(8) MPA vs SPA 七维对比 [p.38]